

Exercices supplémentaires semaine 11

Sophie Baillargeon, Université Laval

24 mars 2021

Question 1

Le package `stats` ne contient pas de méthode `print` pour afficher joliment les résultats de la fonction `optim`. Écrivez une telle méthode `print`, qui produit les impressions suivantes :

```
quadratique <- function(x){
  10*x[1]^2 + 7*x[2]^2 + 4*x[1]*x[2] - 5*x[1] + 7*x[2] + 3
}

test1 <- optim(c(1,1), quadratique)
class(test1) <- "optim"
print(test1)

## L'algorithme a convergé.
## Valeur optimale des paramètres : 0.3712649 -0.6062561
## Valeur de la fonction optimisée en ces paramètres : -0.04924217

test2 <- optim(c(1,1), quadratique, control = list(maxit = 10))
class(test2) <- "optim"
print(test2)

## ***L'algorithme n'a pas convergé.***
## Valeur optimale des paramètres : 1.2625 0.25
## Valeur de la fonction optimisée en ces paramètres : 16.07656
```

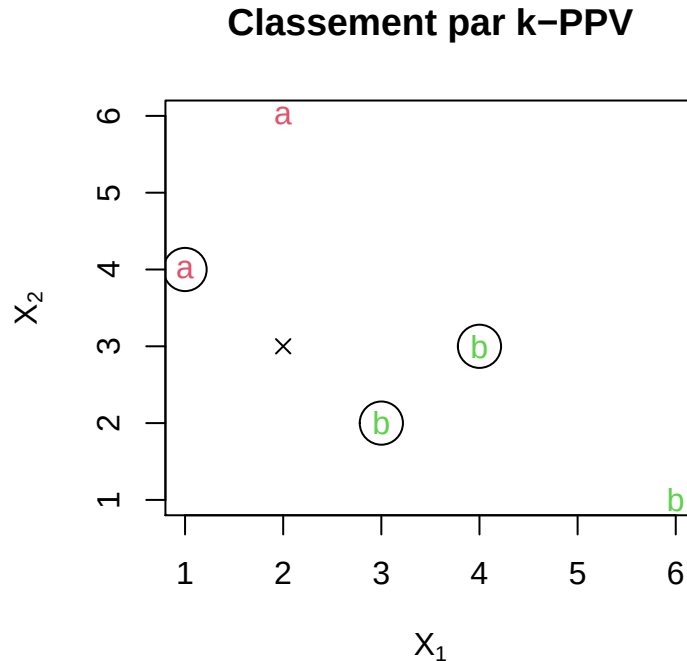
Question 2

Écrivez une fonction qui fait du classement de données par la méthode des k -plus proches voisins (k -PPV). Cette méthode utilise une banque de données pour laquelle on connaît la classe à laquelle appartient chaque observation. On l'appellera banque de données d'entraînement. Les observations de cette banque sont composées de mesures pour un certain nombre de variables. Les données à classer doivent être composées de mesures pour les mêmes variables. L'algorithme derrière la méthode est très simple. Il suffit de calculer, pour chacune des données à classer, la distance entre la donnée et toutes les observations de la banque de données d'entraînement. On identifie ensuite les k plus proches observations, selon une distance euclidienne. k est un paramètre de la méthode dont on doit fixer la valeur à priori. On attribue à la donnée à classer la classe la plus fréquente parmi ces k plus proches observations.

Exemple Par exemple, disons que nous possédons la banque de données d'entraînement suivante :

```
##      X1 X2 | classe
##      1  4 |      a
##      2  6 |      a
##      3  2 |      b
##      4  3 |      b
##      6  1 |      b
```

Ces données contiennent des mesures pour deux variables : X_1 et X_2 . Disons que l'on veut classer la donnée $\mathbf{x} = (X_1 = 2, X_2 = 3)$ et que l'on fixe la valeur de k à 3. Voici un graphique présentant les données d'entraînement ainsi que la donnée à classer.



Les 3 plus proches voisins de $\mathbf{x}$ ont été encadrés. Parmi ces plus proches observations, deux font parties de la classe **b** et une de la classe **a**. Ainsi, on attribue à $\mathbf{x}$ la classe **b**, qui est plus fréquente parmi ses k plus proches voisins.

Instructions de programmation Voici des instructions à suivre dans la programmation de votre fonction.

a) Votre fonction doit porter le nom **kPPV**.

b) Les arguments de votre fonction doivent être :

- **x** : matrice d'observations à classer : chaque ligne est une observation, chaque colonne est une variable.
- **obs** : matrice des observations de la banque de données d'entraînement : chaque ligne est une observation, chaque colonne est une variable.
- **cl.obs** : vecteur identifiant la classe de chacune des observations dans **obs**.
- **k** : nombre de plus proches voisins à considérer (par défaut 1).

c) Votre fonction doit générer une erreur si :

- Les éléments dans la matrice **x** ne sont pas numériques.
- Les éléments dans la matrice **obs** ne sont pas numériques.
- **x** et **obs** ne contiennent pas le même nombre de variables.
- **obs** et **cl.obs** ne contiennent pas le même nombre d'observations.
- **k** prend une valeur inférieure à 1 ou supérieure au nombre d'observations dans **obs**.

d) Description détaillée de l'algorithme à programmer.

Pour chaque observation de la matrice **x** de données à classer, faire les étapes suivantes :

1. Calculer la distance euclidienne entre l'observation à classer et toutes les observations de la banque de données d'entraînement `obs`. Rappel : la distance euclidienne entre les observations $o1$ et $o2$ est $\sqrt{\sum_{i=1}^p (x_i^{o1} - x_i^{o2})^2}$ où x_i^o est la valeur de la variable X_i pour l'observation o et p est le nombre de variables présentes dans les données.
2. Arrondir les distances à 10 chiffres après la virgule.
3. Identifier les k plus proches voisins. Si jamais plus d'une observation est à la $k^{\text{ième}}$ plus petite distance, il faut inclure toutes les observations à cette distance dans le groupe des plus proches voisins. Ainsi, le groupe des plus proches voisins peut contenir plus de k observations.
4. Identifier la classe la plus fréquente parmi les observations du groupe des plus proches voisins. Si jamais plus d'une classe est associée à la plus haute fréquence (il y a une égalité en tête), il faut tirer au hasard une classe parmi les classes les plus fréquentes.
5. Retourner la classe identifiée au point précédent, ainsi que la proportion des observations du groupe des plus proches voisins appartenant à cette classe.

e) La sortie de votre fonction doit être une liste contenant les éléments suivants :

- `cl.x` : vecteur identifiant la classe attribuée à chacune des observations à classer.
- `prop` : vecteur de la proportion d'observations du groupe des plus proches voisins appartenant à la classe attribuée, pour chacune des observations à classer. Cette proportion est une mesure de confiance envers le classement suggéré.

Tests

Pour tester votre fonction, vous pouvez d'abord utiliser les mini-données de l'exemple en introduction. Vous devriez aussi comparer les résultats de votre fonction à ceux de la fonction `knn` (pour le nom anglais de la méthode, soit *k*-nearest neighbors) du package `class`. Votre fonction `kPPV` devrait retourner le même classement que la fonction `knn`, sauf en cas de sélection aléatoire de la classe attribuée.

L'exemple dans la fiche d'aide de la fonction `knn` est très intéressant. Il utilise les données des iris de Fisher, que vous connaissez déjà bien. Avec $k = 3$, pour la 59^{ième} observation à classer (ligne 59 de la matrice `test`), on rencontre un cas où le groupe des plus proches voisins doit être de taille supérieure à k , car plus d'une observation de la banque de données d'entraînement se trouve à la $k^{\text{ième}}$ plus petite distance. De plus, pour ce même cas, plus d'une classe est associée à la plus haute fréquence. La classe attribuée doit donc être sélectionnée au hasard.

Question 3

Imaginons que le vecteur suivant représente les résultats de 15 parties de serpents et échelles entre deux joueurs (1 signifie le premier joueur a gagné, 2 signifie que le deuxième joueur a gagné).

```
gagnants <- c(1, 1, 2, 2, 2, 2, 1, 1, 2, 1, 2, 1, 1, 1, 2)
gagnants
```

```
## [1] 1 1 2 2 2 2 1 1 2 1 2 1 1 1 2
```

Voici une fonction qui devrait produire des statistiques descriptives sur les longueurs des séquences de victoires de chacun des joueurs.

```
## Statistiques descriptives sur des longueurs de séquences
##
## Fonction pour calculer des statistiques descriptives sur des longueurs de séquences
## de victoires (nombres de parties consécutives gagnées par le même joueur)
##
## @param x vecteur dont chaque élément identifie le gagnant d'une partie d'un jeu,
##          la longueur du vecteur est égale au nombre de parties jouées
## @param stat une chaîne de caractères identifiant en français le nom de la
##            statistique descriptive à calculer, soit "moyenne" (valeur par
##            défaut), "médiane" ou "écart-type"
##
## @return un data frame présentant la statistique demandée, calculée sur les
##          longueurs des séquences de victoires, et ce, pour chaque joueur identifié
##          dans le paramètre x
##
statsequences <- function(x, stat = c("moyenne", "médiane", "écart-type")){
  stat <- match.arg(stat)
  fct <- if (stat == "médiane") "median()" else if (stat == "écart-type") "sd()" else "mean()"
  sequences <- rle(x)
  resagg <- aggregate(x = sequences$lengths, by = list(sequences$values), FUN = fct)
  colnames(resagg) <- c("joueur", stat)
  resagg
}
```

Cette fonction comporte un bogue. Lorsqu'appelée comme suit

```
statsequences(x = gagnants, stat = "écart-type")
```

elle devrait produire le résultat suivant

```
##   joueur écart-type
## 1      1  0.8164966
## 2      2  1.5000000
```

alors qu'elle produit plutôt

```
## Error in get(as.character(FUN), mode = "function", envir = envir) :
##   object 'sd()' of mode 'function' was not found
```

Déboguez cette fonction afin qu'elle produise le résultat escompté.

Contrainte : Vous pouvez modifier le corps de la fonction, mais pas la liste des arguments. Ainsi, le nom des arguments et leurs valeurs par défaut ne peuvent pas être modifiés.

Si vous en voulez encore plus :

Question supplémentaire

Cette question vous fait modifier une fonction R permettant de simuler une partie du jeu de serpents et échelles, à 2 joueurs.

Le matériel de ce jeu de société classique est un dé, des pions et une planche de jeu. Voici deux planches de jeux utilisées dans les fonctions présentées ici.



Voici la documentation pour la fonction (sous forme de commentaires `roxygen2`, qui seront vus au prochain cours).

```
#' Simulation d'une partie de serpents et échelles
#'
#' Fonction pour simuler une partie de serpents et échelles à 2 joueurs
#'
#' But du jeu : Arriver le premier sur la dernière case de la planche de jeu.
#'
#' Départ : Tous les joueurs commencent la partie sur la case précédant la case 1 (ou en
#'   dehors de la planche de jeu si celle-ci ne contient pas de case précédant la case 1).
#'
#' Règle du jeu :
#' Pour commencer, le plus jeune des joueurs lance le dé et avance son pion du
#' nombre indiqué sur le dé.
#'   - Si le pion arrive sur le bas d'une échelle, on monte dans la case où il y a
#'     le haut de l'échelle.
#'   - Si le pion arrive sur la queue d'un serpent, on descend dans la case où il y a
#'     la tête du serpent.
#'   - Si le pion arrive sur une case normale, on ne bouge pas.
#' Chaque joueur joue à tour de rôle.
#'
#' Règles simplificatrices qui diffèrent de la version classique du jeu :
#' - Si le pion arrive sur une case déjà occupée, on ne retourne pas au départ.
#' - Pour gagner, il n'est pas nécessaire d'arriver pile sur la dernière case de
#'   la planche de jeu.
#'
#' @param choixplanche identifiant de la planche de jeu à utiliser, soit
#'   "Milton Bradley" (valeur par défaut) ou "toctoc"
#' @param maxtours nombre maximum de tours pouvant être effectués dans la partie
#'
```

```
#' @return \item{partie}{matrice décrivant le déroulement de la partie,
#'          chaque ligne représente le tour de jeu d'un seul joueur,
#'          la 1er colonne contient le numéro du tour de jeu,
#'          la 2e colonne contient le numéro du joueur lançant le
#'          dé (1 pour le premier joueur, 2 pour l'autre joueur),
#'          la 3e colonne contient le résultat du lancé du dé,
#'          la 4e colonne contient la position du joueur sur la
#'          planche de jeu (numéro de la case où se situe le pion
#'          du joueur) à la fin du tour.}
#' @return \item{gagnant}{numéro du joueur gagnant, soit 1 ou 2. Une valeur de
#'          NA signifie que la partie s'est terminée avant qu'un
#'          des joueurs ait atteint la dernière case de la planche
#'          de jeu en raison d'un nombre maximal de tours atteint.}
```

Dans les images précédentes, la planche de gauche est la planche de jeu de l'édition la plus connue du jeu selon [Wikipédia](#), soit la planche du jeu *Chutes and Ladders* commercialisé par la compagnie Milton Bradley (appartenant maintenant à la compagnie Hasbro). En R, représentons cette planche de jeu par le vecteur suivant :

```
MB <- c(37,0,0,10,0,0,0,0,22,rep(0,6),-10,0,0,0,0,21,rep(0,6),56,rep(0,7),8,rep(0,10),
        -21,0,-38,0,16,0,0,0,0,-3,rep(0,5),-43,0,-4,rep(0,6),20,rep(0,8),20,rep(0,6),
        -63,rep(0,5),-20,0,-20,0,0,-20,0,0)
```

MB

```
## [1] 37 0 0 10 0 0 0 0 22 0 0 0 0 0 0 -10 0 0
## [19] 0 0 21 0 0 0 0 0 0 56 0 0 0 0 0 0 0 8
## [37] 0 0 0 0 0 0 0 0 0 0 0 -21 0 -38 0 16 0 0
## [55] 0 -3 0 0 0 0 0 -43 0 -4 0 0 0 0 0 0 20 0
## [73] 0 0 0 0 0 0 0 20 0 0 0 0 0 0 -63 0 0
## [91] 0 0 -20 0 -20 0 0 -20 0 0
```

Dans ce vecteur, un 0 à la position *i* signifie que la case *i* de la planche de jeu ne présente pas de base d'échelle ou de queue de serpent. Un nombre positif *a* à la position *i* signifie que la case *i* présente une base d'échelle qui fait avancer de *a* cases. Finalement, un nombre négatif *-a* à la position *i* signifie que la case *i* présente une queue de serpent qui fait reculer de *a* cases. Le vecteur contient autant d'éléments qu'il y a de cases dans la planche de jeu.

La planche de droite est celle du jeu de mes enfants, soit la planche du jeu *Toc Toc Toc à la rescousse de la grubule* commercialisé par la compagnie Gladius. En R, représentons cette planche de jeu par le vecteur suivant (qui suit la même logique que le vecteur précédent) :

```
ttt <- c(0,0,14,0,0,0,0,0,0,-9,0,0,0,14,0,0,0,0,-13,0,0,0,0,9,
        0,0,0,0,0,13,0,0,0,0,0,-14,0,0,-5,0,5,0,0,0,0,0,-15,0,0)
```

ttt

```
## [1] 0 0 14 0 0 0 0 0 0 -9 0 0 0 14 0 0 0 0 0
## [20] -13 0 0 0 0 9 0 0 0 0 0 13 0 0 0 0 0 -14 0
## [39] 0 -5 0 5 0 0 0 0 0 -15 0 0
```

Voici la fonction.

```
serpentechelle1 <- function(choixplanche = c("Milton Bradley", "totoctoc"),
                             maxtours = length(planche)){

  # Validation des valeurs données en entrée à choixplanche
  choixplanche <- match.arg(choixplanche)
```

```

# Définition de la planche de jeu selon choixplanche
if (choixplanche == "toctoc") {
  planche <- c(0,0,14,0,0,0,0,0,-9,0,0,0,14,0,0,0,0,-13,0,0,0,9,
              0,0,0,0,0,13,0,0,0,0,-14,0,0,-5,0,5,0,0,0,0,-15,0,0)
} else {
  planche <- c(37,0,0,10,0,0,0,0,22,rep(0,6),-10,0,0,0,0,21,rep(0,6), 56,rep(0,7),8,
              rep(0,10),-21,0,-38,0,16,0,0,0,0,-3,rep(0,5),-43,0,-4,rep(0,6),20,
              rep(0,8),20,rep(0,6),-63,rep(0,5),-20,0,-20,0,0,-20,0,0)
}

# Validation des valeurs données en entrée à maxtours
maxtours <- as.integer(maxtours[1])
if (is.na(maxtours) || maxtours <= 0) stop("'maxtours' doit etre un entier positif")

# Initialisation de la matrice contenant le descriptif
# complet du déroulement de la partie simulée
partie <- matrix(NA, nrow = 2*maxtours, ncol = 4)
colnames(partie) <- c("tour", "joueur", "de", "position")
partie[, "tour"] <- rep(1:(nrow(partie)/2), each = 2)

# Boucle simulant la partie
i <- 0
joueur <- 1
positions <- c(0, 0) # position 0 pour tous au début de la partie

repeat{ # une répétition = un tour pour un joueur

  if (i >= 2*maxtours) {
    warning("nombre maximal de tours atteint")
    break
    # la partie est terminée si le nombre maximum de tours est atteint
  }
  i <- i + 1

  # lancé du dé
  de <- sample(1:6, 1)

  # déplacement du pion du joueur
  postempo <- positions[joueur] + de
  ajout <- if(postempo <= length(planche)) planche[postempo] else 0
  positions[joueur] <- postempo + ajout

  # enregistrement du résultat
  if (positions[joueur] >= length(planche)) {
    partie[i, -1] <- c(joueur, de, length(planche))
    break
    # si le joueur a atteint la dernière case de la planche de jeu,
    # la partie est terminée
  } else {
    partie[i, -1] <- c(joueur, de, positions[joueur])
  }

  # changement de joueur

```

```

    joueur <- 3 - joueur # joueur = 1 devient joueur = 2 et joueur = 2 devient joueur = 1
  }

  # Sortie
  gagnant <- if (i == 2*maxtours) NA else partie[i, "joueur"]
  out <- list(partie = partie[1:i, ], gagnant = gagnant)
  class(out) <- "se"
  out
}

```

Voici une méthode pour la fonction générique `print` permettant de contrôler l’affichage d’un objet de classe `se`, produit par n’importe laquelle des trois fonctions.

```

print.se <- function(x, ...){

  if (is.na(x$gagnant)){
    cat("La partie s'est terminée avant qu'un joueur gagne la partie.\n")
  } else {
    cat("Le joueur", x$gagnant, "a gagné la partie.\n")
  }
}

```

Voici quelques exemples d’appels de cette fonction, accompagnés du résultat affiché.

```
serpenteche1(choixplanche = "Milton Bradley")
```

```
## Le joueur 2 a gagné la partie.
```

```
serpenteche1(choixplanche = "Milton Bradley", maxtours = 10)
```

```
## Warning in serpenteche1(choixplanche = "Milton Bradley", maxtours = 10):
## nombre maximal de tours atteint
```

```
## La partie s'est terminée avant qu'un joueur gagne la partie.
```

```
serpenteche1(choixplanche = "toctoctoc")
```

```
## Le joueur 2 a gagné la partie.
```

Reprenez le code de la fonction `serpenteche1` pour créer une nouvelle fonction, nommée `serpenteche`, qui sera une version modifiée de `serpenteche1`. La documentation de cette nouvelle fonction sous forme de commentaires `roxygen2` est la suivante .

```

#' Simulation d'une partie de serpents et échelles
#'
#' Fonction pour simuler une partie de serpents et échelles
#'
#' But du jeu : Arriver le premier sur la dernière case de la planche de jeu.
#'
#' Départ : Tous les joueurs commencent la partie sur la case précédant la case 1 (ou en
#'   dehors de la planche de jeu si celle-ci ne contient pas de case précédant la case 1).
#'
#' Règle du jeu :
#'
#' Pour commencer, le plus jeune des joueurs lance le dé et avance son pion du
#' nombre indiqué sur le dé.
#' - Si le pion arrive sur le bas d'une échelle, on monte dans la case où il y a

```



```

#'      le haut de l'échelle.
#'      - Si le pion arrive sur la queue du serpent, on descend dans la case où il y a
#'      la tête du serpent.
#'      - Si le pion arrive sur une case normale, on ne bouge pas.
#'      - Si le pion arrive sur une case déjà occupée, on retourne au départ.
#' Chaque joueur joue à tour de rôle.
#' Pour gagner, il faut arriver pile sur la dernière case de la planche de jeu,
#' sinon on recule du nombre de points restants.
#'
#' @param choixplanche identifiant de la planche de jeu à utiliser, soit
#'      "Milton Bradley" (valeur par défaut), "toctoc" ou "fournie"
#' @param planche vecteur caractérisant une planche de jeu à fournir si
#'      \code{choixplanche = "fournie"}
#' @param njoueurs nombre de joueurs, entier entre 2 (valeur par défaut) et 4
#' @param maxtours nombre maximum de tours pouvant être effectués dans la partie
#'
#' @return \item{partie}{matrice décrivant le déroulement de la partie,
#'      chaque ligne représente le tour de jeu d'un seul joueur,
#'      la 1er colonne contient le numéro du tour de jeu,
#'      la 2e colonne contient le numéro du joueur lançant le
#'      dé (1 pour le premier joueur, 2 pour l'autre joueur),
#'      la 3e colonne contient le résultat du lancé du dé,
#'      la 4e colonne contient la position du joueur sur la
#'      planche de jeu (numéro de la case où se situe le pion
#'      du joueur) à la fin du tour.}
#' @return \item{gagnant}{numéro du joueur gagnant, soit 1 ou 2. Une valeur de
#'      NA signifie que la partie s'est terminée avant qu'un
#'      des joueurs ait atteint la dernière case de la planche
#'      de jeu en raison d'un nombre maximal de tours atteint.}

```

Les modifications dans cette fonction par rapport à `serpentechelle1` sont décrites dans les points suivants. Vous devez produire la fonction modifiée `serpentechelle`.

a)

L'argument `choixplanche` a une nouvelle valeur possible, soit `"fournie"`. Lorsque la valeur `"fournie"` est donnée en entrée à `choixplanche`, la planche de jeu est déterminée par l'argument `planche`.

b)

La fonction a un nouvel argument, nommé `planche`, qui permet de fournir en entrée à la fonction un vecteur décrivant une planche de jeu. La valeur par défaut de ce paramètre doit être `NULL`.

Le vecteur `planche` sera utilisé comme planche de jeu uniquement si `choixplanche = "fournie"`. Dans ce cas (et dans ce cas seulement), la valeur donnée en entrée à `planche` doit être validée. Vous devez alors faire une conversion automatique de la valeur donnée en entrée à `planche` en vecteur atomique. Ensuite, vous devez provoquer l'arrêt de la fonction si `planche` prend la valeur `NULL` ou si les éléments de `planche` ne sont pas tous des valeurs entières.

Si `choixplanche` prend une autre valeur que `"fournie"`, mais qu'une valeur a tout de même été donnée en entrée à l'argument `planche`, vous devez émettre un avertissement communiquant à l'utilisateur que l'argument `planche` qu'il a fourni est inutilisé, car `choixplanche` ne prend pas la valeur `"fournie"`.

c)

La fonction a un autre nouvel argument, nommé `njoueurs`, qui sert à déterminer le nombre de joueurs dans la partie. La valeur par défaut de ce paramètre doit être 2.

Vous devez faire une conversion automatique de la valeur donnée en entrée à `njoueurs` en valeur entière et conserver uniquement le premier élément si un objet comprenant plus d'un élément a été donné en entrée. Ensuite, vous devez provoquer l'arrêt de la fonction si `njoueurs` prend une valeur autre que 2, 3 ou 4.

Ensuite, vous devez modifier le corps de la fonction afin de simuler une partie à `njoueurs` joueurs. La fonction `serpenteche1` était uniquement conçue pour simuler des parties à 2 joueurs.

d)

La règle de jeu suivante a été modifiée :

Si le pion arrive sur une case déjà occupée, on retourne au départ.

Vous devez adapter le code de façon à respecter cette nouvelle règle dans les simulations.

e)

La règle de jeu suivante a été modifiée :

Pour gagner, il faut arriver pile sur la dernière case de la planche de jeu, sinon on recule du nombre de points restants.

Vous devez adapter le code de façon à respecter cette nouvelle règle dans les simulations.

f)

Simulez un grand nombre de parties à 2 joueurs (disons 20000) avec votre nouvelle fonction `serpenteche1`, et ce, pour chacune des planches de jeu ("Milton Bradley" et "toctoc"). Est-ce que les chances de gagner des joueurs 1 et 2 sont différentes de ce que vous avez obtenu à la question 4 a) ?

g)

Écrivez une méthode pour la fonction générique `summary` pour un objet retourné par la fonction `serpenteche1` (donc un objet de classe "`se`"). Cette méthode devrait produire l'affichage illustré dans les exemples suivants.

```
summary.se <- function(x, ...) {  
  
  print(x)  
  cat("La partie a duré", x$partie[nrow(x$partie), "tour"], "tours.\n\n")  
  
  # déplacements par tour  
  for (i in unique(x$partie[, "joueur"])){  
    dep <- diff(c(0, x$partie[x$partie[, "joueur"] == i, "position"]))  
    cat("Le joueur", i, "s'est déplacé en moyenne de", mean(dep), "cases par tour\n")  
  }  
}
```

```
exemple1 <- serpenteche1()  
summary(exemple1)
```

```
exemple1 <- serpenteche1()  
summary(exemple1)
```

```
## Le joueur 2 a gagné la partie.  
## La partie a duré 67 tours.  
##  
## Le joueur 1 s'est déplacé en moyenne de 1.223881 cases par tour  
## Le joueur 2 s'est déplacé en moyenne de 1.492537 cases par tour
```

```
exemple2 <- serpentechelle()  
summary(exemple2)
```

```
exemple2 <- serpentechelle1()  
summary(exemple2)
```

```
## Le joueur 2 a gagné la partie.  
## La partie a duré 27 tours.  
##  
## Le joueur 1 s'est déplacé en moyenne de 2.851852 cases par tour  
## Le joueur 2 s'est déplacé en moyenne de 3.703704 cases par tour
```